

Gestor de "Cuentas Claras"

1. Visión General del Proyecto

Descripción: Aplicación web estática de ejecución local (*offline-first*), diseñada exclusivamente para dispositivos móviles. **Objetivo Principal:** Reemplazar el seguimiento de gastos en hojas de cálculo tradicionales mediante una interfaz ágil que automatice el cálculo de saldos ("Complemento") basándose en el orden cronológico real de las transacciones, facilitando la generación de reportes visuales (capturas de pantalla) claros y auditables.

2. Arquitectura Tecnológica

Al ser un proyecto que no dependerá de servidores ni de conexión a internet, el *stack* tecnológico se mantiene estrictamente del lado del cliente:

- **Lógica y Control:** Vanilla JavaScript (ES6+). Toda la lógica de reordenamiento y cálculos matemáticos se ejecutará en el navegador.
- **Estructura:** HTML5 semántico en un único archivo (index.html) para facilitar la portabilidad.
- **Estilos y UI:** Tailwind CSS. Ideal para aplicar clases de utilidad directamente en el HTML, logrando un diseño *mobile-first*, limpio y responsivo sin necesidad de escribir hojas de estilo extensas. Se requerirá usar el archivo precompilado de Tailwind para que funcione *offline*.
- **Persistencia de Datos:** localStorage de la API web para almacenamiento en tiempo real, complementado con la API Blob de JavaScript para la generación de archivos físicos de respaldo.

3. Especificación de la Interfaz de Usuario (UI)

La pantalla se dividirá en tres componentes principales:

3.1. Panel de Ingreso (Sticky Header)

Fijado en la parte superior para estar siempre accesible.

- **Campo "Concepto":** Input de texto (ej. "Aceite para compresor").
- **Campo "Cantidad":** Input de tipo numérico (type="number").
- **Selector "Fecha/Hora":** Input type="datetime-local". Por defecto, captura el Date.now() al momento de abrir la app.
- **Botón de Acción:** "Registrar Gasto" (Botón de ancho completo, fácil de presionar).

3.2. Tabla de Visualización (Data Grid)

El núcleo visual del reporte.

- **Columnas Estrictas:** Concepto | Cantidad | Complemento.
- **Ordenamiento:** Cronológico descendente o ascendente (basado en la fecha ingresada, no en la fecha de captura).
- **Fila de Cabecera:** Muestra el Fondo Inicial (ej. "Cuenta 120k").

3.3. Barra de Herramientas de Control (Footer/Modal)

- **Modo Captura:** Botón que oculta el Panel de Ingreso y la Barra de Herramientas, dejando únicamente la Tabla visible para tomar un *screenshot* limpio. Un toque en cualquier parte de la pantalla restaura la UI.
- **Exportar Respaldo:** Botón que genera un archivo .json y un .csv.
- **Importar Respaldo:** Input type="file" para cargar un .json previo.

4. Estructura de Datos (Esquema JSON)

Para el almacenamiento en localStorage y la exportación, la estructura de la base de datos documental seguirá este formato:

JSON

```
{
  "fondo_activo": {
    "nombre": "Cuenta 120k",
    "monto_inicial": 120000,
    "fecha_creacion": "2026-06-14T16:39:36"
  },
  "transacciones": [
    {
      "id": "1718385000123",
      "concepto": "Pago de don angel",
      "cantidad": 9178,
      "fecha_transaccion": "2026-06-14T10:00:00"
    },
    {
```

```
"id": "1718385050456",  
"concepto": "Pago Emy",  
"cantidad": 2345,  
"fecha_transaccion": "2026-06-12T15:30:00"  
}  
]  
}
```

(Nota: El ID se genera usando un timestamp único de la creación del registro).

5. Lógica de Negocio (Reglas del Sistema)

1. **Regla de Re-cálculo Dinámico:** Cada vez que se inserta, edita o elimina una transacción, el sistema debe tomar el arreglo completo de transacciones, ordenarlo de la fecha más antigua a la más reciente, y ejecutar un ciclo for/reduce.
2. **Fórmula del Complemento:** $\text{Complemento_Actual} = \text{Complemento_Fila_Anterior} - \text{Cantidad_Fila_Actual}$. (En la primera transacción, el Complemento_Fila_Anterior es igual al monto_inicial).
3. **Persistencia Inmediata:** Inmediatamente después del re-cálculo, el nuevo estado de los datos se sobrescribe en el localStorage para evitar pérdida de datos si la aplicación se cierra de golpe.

6. Fases de Desarrollo Sugeridas

- **Fase 1 (Core & UI):** Maquetar HTML + Tailwind. Crear la funcionalidad de agregar elementos a un array temporal y mostrarlos en la tabla.
- **Fase 2 (Motor Matemático):** Programar el algoritmo de ordenamiento por fecha y el cálculo en cascada de la columna "Complemento".
- **Fase 3 (Persistencia):** Conectar el array de datos con localStorage.
- **Fase 4 (I/O Local):** Desarrollar las funciones de la API Blob para la exportación a .json y la lectura mediante FileReader para la importación.
- **Fase 5 (Refinamiento):** Implementar el "Modo Captura" y validaciones de inputs nulos.